Q1. What is a Common Table Expression (CTE), and how does it improve SQL query readability?

A Common Table Expression (CTE) is a temporary named result set that can be defined using the `WITH` clause and used within a SQL query.

CTEs improve SQL query readability because they divide a complex query into smaller, logical, and easier-to-understand parts. Instead of writing nested subqueries, we can define the intermediate result once and then refer to it by name.

Advantages of CTEs:

- Make complex queries easier to read.
- Reduce the use of deeply nested subqueries.
- Can be reused within the same query.
- Useful for recursive queries and hierarchical data.

Q2. Why are some views updatable while others are read-only? Explain with an example.

A view is a virtual table created from one or more tables using a SQL query.

Some views are updatable because the database can clearly identify which row in the underlying table should be modified. Simple views based on a single table are generally updatable.

Other views are read-only when they contain operations that make it difficult to map the result back to individual rows of the base table, such as:

- `GROUP BY`
- Aggregate functions such as `SUM()` or `AVG()`
- `DISTINCT`
- `UNION`
- Certain joins and calculated columns

Q3. What advantages do stored procedures offer compared to writing raw SQL queries repeatedly?

A stored procedure is a precompiled collection of SQL statements stored in the database and executed when required.

Stored procedures offer several advantages over repeatedly writing raw SQL queries:

1. **Code Reusability:**
   A stored procedure can be created once and executed many times.
2. **Reduced Repetition:**
   The same SQL logic does not need to be written repeatedly.
3. **Better Performance:**
   The database can optimize and reuse the execution plan for the procedure.
4. **Improved Security:**
   Users can be given permission to execute a procedure without giving them direct access to the underlying tables.
5. **Centralized Business Logic:**
   Important database operations can be maintained in one place.
6. **Easy Maintenance:**
   Changes can be made inside the stored procedure instead of changing the same query in multiple applications.

Q4. What is the purpose of triggers in a database? Mention one use case where a trigger is essential.

A trigger is a special database program that automatically executes when a specified event occurs on a table, such as:

- `INSERT`
- `UPDATE`
- `DELETE`

The main purpose of triggers is to automatically perform actions in response to changes in database data. They are useful for maintaining data integrity, auditing changes, and automatically updating related information.

One essential use case:

A trigger can be used to archive deleted records**.** Whenever a product is deleted from the `Products` table, the trigger can automatically copy the deleted record into a `ProductArchive` table.

Q5. Explain the need for data modelling and normalization when designing a database.

Data modelling is the process of designing the structure of a database, including tables, columns, relationships, primary keys, and foreign keys.

Normalization is the process of organizing data into properly structured tables to reduce redundancy and improve data consistency.

Need for data modelling:

Data modelling helps to:

- Clearly define how data will be stored.
- Establish relationships between tables.
- Identify primary and foreign keys.
- Improve database organization.
- Make the database easier to maintain and expand.

Need for normalization:

Normalization helps to:

- Reduce duplicate data.
- Prevent data inconsistency.
- Avoid update, insertion, and deletion anomalies.
- Improve data integrity.
- Make the database more efficient and organized.

Q6. Write a CTE to calculate the total revenue for each product (Revenues = Price × Quantity), and return only products where revenue > 3000.

A CTE can be used to first calculate the revenue for each product and then filter the products whose revenue is greater than `3000`.

```
WITH ProductRevenue AS (
  SELECT
    p.ProductID,
    p.ProductName,
    p.Price,
    SUM(s.Quantity) AS TotalQuantity,
    p.Price * SUM(s.Quantity) AS Revenue
  FROM Products p
  JOIN Sales s
    ON p.ProductID = s.ProductID
```

```
    GROUP BY
        p.ProductID,
        p.ProductName,
        p.Price
)
SELECT
    ProductID,
    ProductName,
    Price,
    TotalQuantity,
    Revenue
FROM ProductRevenue
WHERE Revenue > 3000;
```

Q7. Create a view named vw_CategorySummary that shows: Category, TotalProducts, AveragePrice.

The `vw_CategorySummary` view groups products by category and calculates the total number of products and the average price for each category.

```
CREATE VIEW vw_CategorySummary AS
SELECT
    Category,
    COUNT(*) AS TotalProducts,
    AVG(Price) AS AveragePrice
FROM Products
GROUP BY Category;
```

Q8. Create an updatable view containing ProductID, ProductName, and Price. Then update the price of ProductID = 1 using the view.

Since this view is based directly on a single table and does not contain grouping or aggregate functions, it can be updated.

Step 1: Create the view:-

```
CREATE VIEW vw_ProductDetails AS
SELECT
   ProductID,
   ProductName,
   Price
FROM Products;
```

Step 2: Update the price of ProductID = 1

The assignment does not specify a new price, so the following uses 1500 as an example:

```
UPDATE vw_ProductDetails
SET Price = 1500
WHERE ProductID = 1;
```

Step 3: Verify the updated record

```
SELECT *
FROM vw_ProductDetails
WHERE ProductID = 1;
```

This update is performed through the view and changes the corresponding record in the underlying `Products` table.

Q9. Create a stored procedure that accepts a category name and returns all products belonging to that category.

A stored procedure can accept the category name as an input parameter and return all products belonging to that category.

SQL Query:

```
DELIMITER //



CREATE PROCEDURE GetProductsByCategory(

   IN category_name VARCHAR(50)

)

BEGIN

   SELECT

      ProductID,

      ProductName,

      Category,

      Price

   FROM Products

   WHERE Category = category_name;

END //



DELIMITER ;
```

To execute the procedure:

```
CALL GetProductsByCategory('Electronics');
```

Q10. Create an AFTER DELETE trigger on the Products table that archives deleted product rows into a new table ProductArchive. The archive should store ProductID, ProductName, Category, Price, and DeletedAt timestamp.

First, we create the ProductArchive table. Then we create an AFTER DELETE trigger that automatically copies the deleted product information into the archive table.

Step 1: Create the archive table

```
CREATE TABLE ProductArchive (
   ProductID INT,
   ProductName VARCHAR(100),
   Category VARCHAR(50),
   Price DECIMAL(10,2),
   DeletedAt TIMESTAMP
);
```

Step 2: Create the AFTER DELETE trigger

```
DELIMITER //

CREATE TRIGGER trg_AfterDeleteProduct
AFTER DELETE ON Products
FOR EACH ROW
BEGIN
   INSERT INTO ProductArchive
   (
      ProductID,
      ProductName,
      Category,
```

```
    Price,
    DeletedAt
  )
  VALUES
  (
    OLD.ProductID,
    OLD.ProductName,
    OLD.Category,
    OLD.Price,
    CURRENT_TIMESTAMP
  );
END //

DELIMITER ;
```

Step 3: Example of deleting a product

```
DELETE FROM Products
WHERE ProductID = 1;
```

After the deletion, the trigger automatically inserts the deleted product's information into ProductArchive, along with the deletion timestamp.